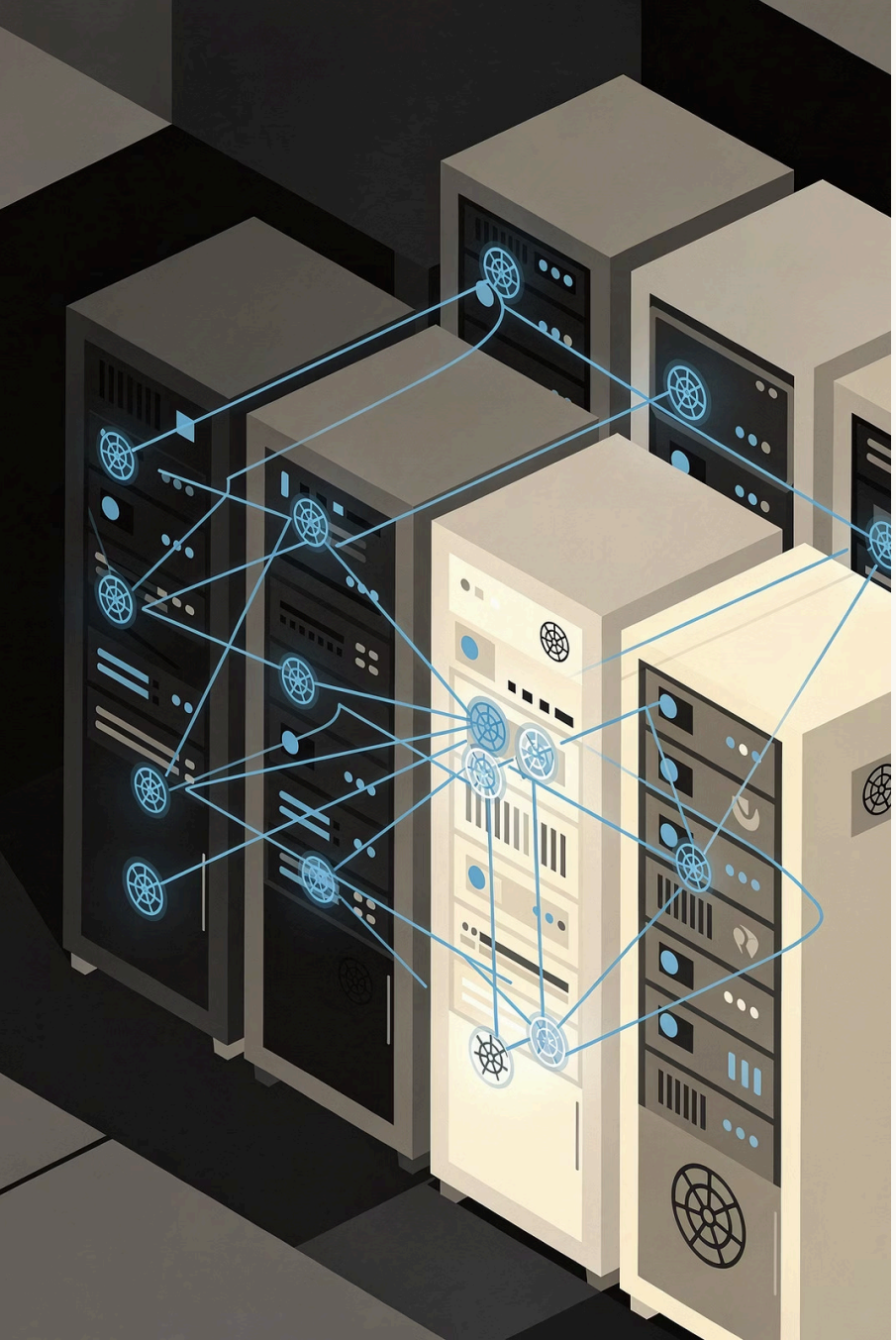


Kubernetes Cluster Installation

A complete, step-by-step guide to bootstrapping a production-ready Kubernetes cluster using **kubeadm v1.34** on Linux virtual machines — covering the Master (Control Plane) node and Worker nodes from scratch.

KUBEADM V1.34 | SELF-MANAGED | LINUX / UBUNTU

What is kubeadm?

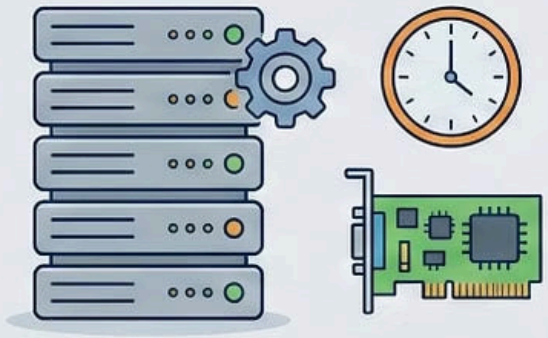
kubeadm is the official Kubernetes bootstrap tool that installs and configures all the control plane components, getting your cluster ready quickly and reliably. It handles the complex initialization tasks so you can focus on deploying workloads.

Along with the core control plane components — **API Server, ETCD, Controller Manager,** and **Scheduler** — kubeadm assists with the installation of essential CLI tools including **kubectl** and **kubelet**.

Note: In this demo, we will be installing Kubernetes on VMs on cloud (Self-Managed).

BUILDING A KUBERNETES CLUSTER WITH KUBEADM:
From Theory to Practice

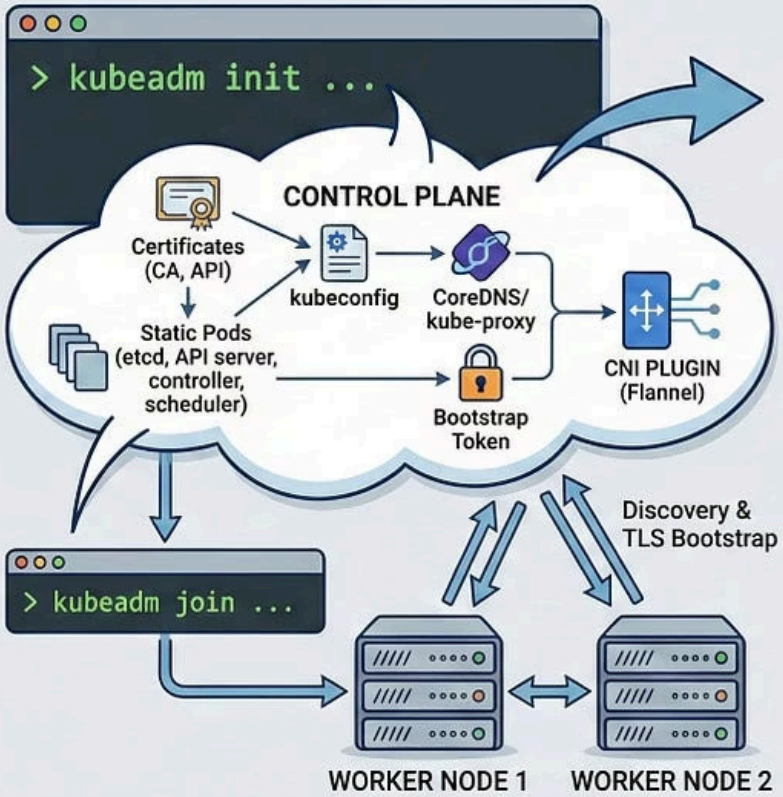
1. THEORY & PREREQUISITES
(The Foundation)



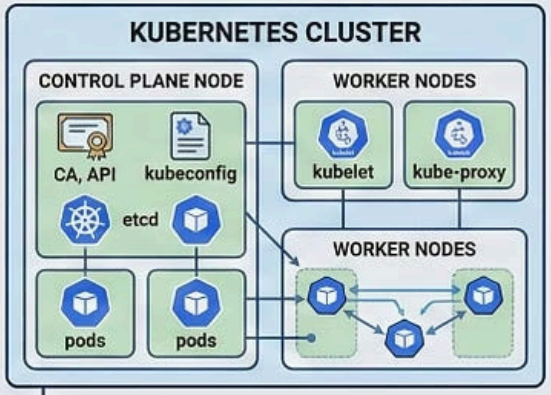
- Kubeadm Philosophy: Bootstrap Only
- VMs, OS Config (cgroupv2, swap off, firewall disabled)
- System Requirements
- Container Runtime: containerd

Not full provisioning, just bootstrap

2. CORE PROCESS
(kubeadm init & join)



3. PRACTICE & VERIFICATION
(The Cluster)



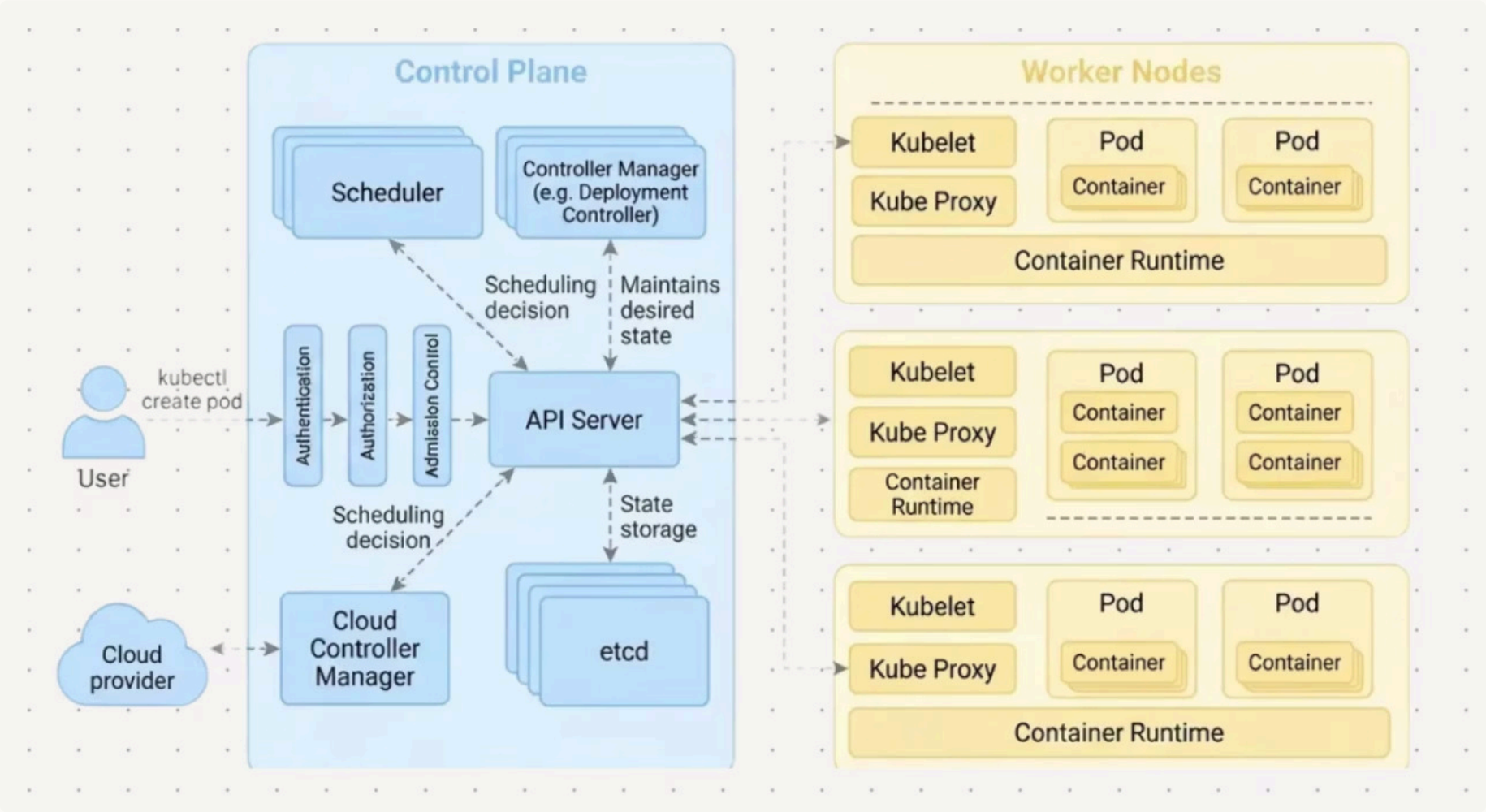
CLUSTER READY

- Node Status (Ready)
- Pod Connectivity
- Routing Tables
- Key Directories (/etc/kubernetes, /var/lib/kubelet)

Troubleshooting Tips & Next Steps

Cluster Architecture Overview

A standard Kubernetes cluster used in this guide consists of **three virtual machines**: one Master Node acting as the Control Plane, and two Worker Nodes that host the actual application Pods. Understanding this layout is essential before executing any commands.



The Control Plane is the brain of the cluster — it manages scheduling, state, and API communications. The Worker Nodes execute the containerized workloads assigned by the control plane, with **kubelet** ensuring containers run as expected and **kube-proxy** handling network rules.

Ways to Install Kubernetes

Before diving into kubeadm, it is worth understanding the installation landscape. Your choice of installation method depends on whether the cluster is for local testing or production, and whether you want a self-managed or fully managed experience.

kubeadm is the official tool designed to bootstrap a Kubernetes cluster. It installs and configures all control plane components — **API Server, ETCD, Controller Manager, and Scheduler** — and prepares the cluster for you to join worker nodes. Along with the control plane components, it assists in installing the essential CLI tools: **kubelet**, **kubectl**, and **kubeadm** itself.

SELF-MANAGED OPTIONS

- **KIND** — Kubernetes IN Docker, great for CI/testing
- **k3s** — Lightweight Kubernetes for edge/IoT
- **Minikube** — Single-node local dev cluster
- **kubeadm on VMs** — Production-like, multi-node (this guide)
- **Vagrant/VirtualBox or Multipass** — Local multi-node labs

MANAGED KUBERNETES ON CLOUD

- **EKS** — Elastic Kubernetes Service (AWS)
- **AKS** — Azure Kubernetes Service (Microsoft)
- **GKE** — Google Kubernetes Engine (Google)

In this guide, we install Kubernetes on **VMs on Cloud (Self-Managed)** using kubeadm — the most widely used approach for learning production Kubernetes operations.

This guide covers: Self-Managed Kubernetes on Cloud VMs using kubeadm — the most common path for learning production-grade cluster operations.

High-Level Setup Flow

1

Setup 3 Cloud VMs

1 Master, 2 Worker Nodes

2

Open Required Ports

Control plane & component ports

3

Run Prerequisite Steps

On both Master & Worker nodes

4

Initialize Control Plane

kubeadm init on Master

5

Install CNI Plugin

Calico networking

6

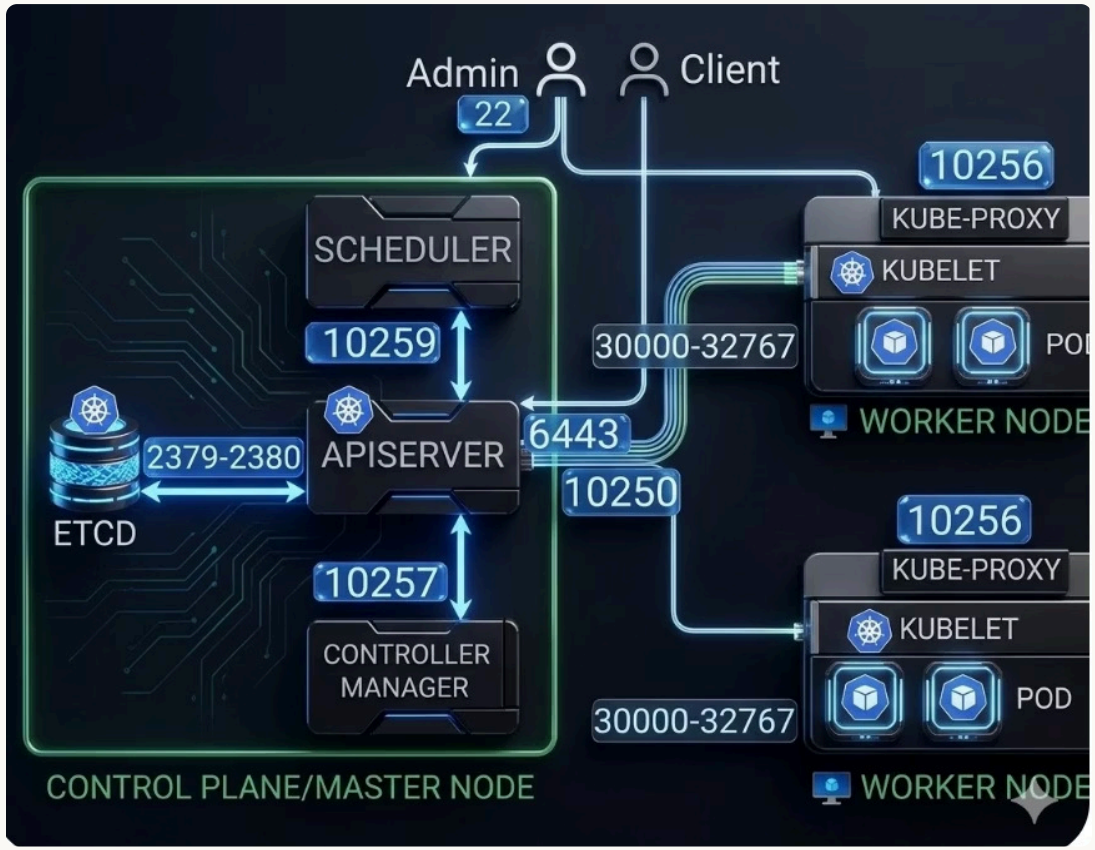
Join Worker Nodes

kubeadm join on each Worker

Required Ports — AWS EC2 Security Groups

If you are running on AWS EC2 (or any cloud VM), you must configure **two security groups** — one for the Master Node and one for the Worker Nodes — and allow specific inbound traffic on the ports below. Refer to the official Kubernetes port reference for the full list.

Control Plane ports: 6443 (API Server), 2379–2380 (ETCD), 10257 (Controller Manager), 10259 (Scheduler), 10250 (Kubelet).
Worker Node ports: 10256 (Kube-Proxy), 10250 (Kubelet), 30000–32767 (NodePort Services).
On AWS EC2, also remember to **disable source/destination check** on the VMs. Reference: **kubernetes.io/docs/reference/networking/ports-and-protocols/**


Admin
Client
22
10256
KUBE-PROXY
KUBELET
POD
WORKER NODE
SCHEDULER
10259
30000-32767
APISERVER
6443
2379-2380
ETCD
10250
10257
CONTROLLER MANAGER
10256
KUBE-PROXY
KUBELET
POD
30000-32767
WORKER NODE
CONTROL PLANE/MASTER NODE

PART 1 — MASTER NODE

Master Node Setup: Overview

All steps in Part 1 are executed exclusively on the **Master (Control Plane) node**. These steps prepare the operating system, install the container runtime and networking components, and finally initialize the Kubernetes control plane using kubeadm. Worker nodes will follow a nearly identical process in Part 2.

01

Steps 1–3: OS Prerequisites

Disable swap, load required kernel modules, configure networking sysctl parameters.

02

Steps 4–6: Container Runtime Stack

Install containerd, runc, and CNI plugins to enable container execution.

03

Steps 7–8: Kubernetes Tooling

Install kubelet, kubeadm, and kubectl; configure crictl.

04

Steps 9–11: Cluster Bootstrap

Initialize control plane, configure kubectl access, install Calico CNI.

MASTER NODE — STEP 1

Disable Swap

Kubernetes requires swap to be **completely disabled** on all nodes. If swap is enabled, the kubelet will refuse to start (by default). Swap usage introduces unpredictable latency for memory-sensitive workloads — Kubernetes prefers to enforce its own resource limits via cgroups instead of relying on OS-level swapping.

Disable swap immediately (current session):

```
swapoff -a
```

Permanently disable swap by commenting out the swap entry in /etc/fstab:

```
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
```

Verify swap is disabled (all values should be 0 under Swap row):

```
free -m
```

Why this matters: The `sed` command comments out any line in `/etc/fstab` containing the word "swap", ensuring the change survives a reboot. Always confirm with `free -m` that Swap total is 0.

MASTER NODE — STEP 2

Load Kernel Modules

Kubernetes networking relies on two Linux kernel modules: **overlay** (used by the container runtime for layered filesystems) and **br_netfilter** (allows iptables to see bridged traffic, required for pod-to-pod networking). These must be loaded at boot time.

Create the module load configuration file:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

Load the modules immediately into the running kernel:

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

Explanation: Writing to `/etc/modules-load.d/k8s.conf` ensures both modules are loaded automatically on every reboot. The `modprobe` commands activate them immediately without requiring a restart.

MASTER NODE — STEP 3

Configure Kernel Networking Parameters

For Kubernetes networking to function correctly, several kernel-level sysctl parameters must be set. These allow iptables to process bridged network traffic and enable IPv4 forwarding — both critical for Pod communication across nodes.

Write the sysctl configuration file:

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
```

Apply the settings immediately without rebooting:

```
sudo sysctl --system
```

Parameter breakdown: `bridge-nf-call-iptables` ensures bridged IPv4/IPv6 traffic is processed by iptables — essential for network policies. `ip_forward` enables the Linux kernel to route packets between network interfaces, which is required for cross-node pod communication.

MASTER NODE — STEP 4

Install Container Runtime: containerd

containerd is the industry-standard container runtime used by Kubernetes. It manages the full container lifecycle — image pulling, snapshot management, container execution, and supervision. We install it directly from the official GitHub release tarball to get a known-good version.

Download and extract the containerd binary:

```
curl -LO https://github.com/containerd/containerd/releases/download/v1.7.14/containerd-1.7.14-linux-amd64.tar.gz
sudo tar Cxzvf /usr/local containerd-1.7.14-linux-amd64.tar.gz
```

Install the systemd service unit file:

```
curl -LO https://raw.githubusercontent.com/containerd/containerd/main/containerd.service
sudo mkdir -p /usr/local/lib/systemd/system/
sudo mv containerd.service /usr/local/lib/systemd/system/
```

Generate default configuration and enable SystemdCgroup:

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
```

Enable and start the containerd service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now containerd
```

Why SystemdCgroup = true? Kubernetes manages cgroups via systemd. If containerd uses a different cgroup driver, resource tracking becomes inconsistent. Setting `SystemdCgroup = true` aligns both systems to use the same cgroup hierarchy.

MASTER NODE — STEP 5

Install runc

runc is the low-level OCI-compliant container runtime that containerd delegates to for actually spawning container processes. It directly calls Linux kernel features — namespaces, cgroups, seccomp — to create isolated container environments. Without runc, containerd cannot run containers.

Download runc and install it as a system binary:

```
curl -LO https://github.com/opencontainers/runc/releases/download/v1.1.12/runc.amd64
sudo install -m 755 runc.amd64 /usr/local/sbin/runc
```

Architecture note: The `install -m 755` command copies the binary to `/usr/local/sbin/runc` with executable permissions. This is the standard location that containerd looks for runc when spawning containers. Always use the `amd64` binary for x86-64 VMs.

MASTER NODE — STEP 6

Install CNI Plugins

CNI (Container Network Interface) plugins provide the low-level networking primitives that higher-level CNI solutions like Calico use to configure pod networking. They handle tasks like assigning IP addresses to pods and setting up virtual network interfaces inside containers.

Download the CNI plugin archive:

```
curl -LO https://github.com/containernetworking/plugins/releases/download/v1.5.0/cni-plugins-linux-amd64-v1.5.0.tgz
```

Create the standard CNI binary directory and extract plugins:

```
sudo mkdir -p /opt/cni/bin
sudo tar Cxzvf /opt/cni/bin cni-plugins-linux-amd64-v1.5.0.tgz
```

Why /opt/cni/bin? This is the default path that containerd and Kubernetes look for CNI plugin binaries. Calico and other CNI providers expect the base plugins to already exist here before they install their own components.

MASTER NODE — STEP 7

Install Kubernetes Components

This step installs the three core Kubernetes binaries: **kubelet** (the node agent that manages pods), **kubeadm** (the cluster bootstrap tool), and **kubectl** (the command-line interface). We add the official Kubernetes apt repository for v1.34 and pin the versions to prevent accidental upgrades.

Install prerequisites for the apt repository:

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

Add the Kubernetes apt signing key and repository:

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
```

Install kubelet, kubeadm, and kubectl, then hold versions:

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Verify successful installation:

```
kubeadm version
kubectl version --client
```

apt-mark hold prevents these packages from being automatically upgraded during a routine `apt upgrade`. Kubernetes components must be upgraded in a controlled, sequential manner — never accidentally.

MASTER NODE — STEP 8

Configure crictl

crictl is the CRI (Container Runtime Interface) command-line tool used to inspect and debug containers at the runtime level — similar to `docker` CLI but for CRI-compatible runtimes like containerd. It must be pointed at the correct runtime socket to work properly.

Set the containerd socket as the default runtime endpoint for crictl:

```
sudo crictl config runtime-endpoint unix:///var/run/containerd/containerd.sock
```

Why configure this? Without pointing `crictl` to the correct socket, commands like `crictl ps` or `crictl pods` will fail or produce warnings. This is especially important for debugging pod startup issues at the container runtime level, below what `kubectl` can see.

MASTER NODE — STEP 9

Initialize the Control Plane

This is the most critical step on the Master node. `kubeadm init` bootstraps the entire Kubernetes control plane — it generates TLS certificates, starts the API server, controller manager, scheduler, and etcd, and creates the initial cluster state. Replace `<MASTER_IP>` with the actual private IP of your master VM.

Initialize the control plane:

```
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \
  --apiserver-advertise-address=<MASTER_IP> \
  --node-name master
```

After initialization completes successfully, kubeadm will print a **join command** at the bottom of its output. This command contains a unique token and certificate hash. **Copy and save this command** — you will need it in Part 2 to join the worker nodes.

The join command will look similar to:

```
kubeadm join 172.31.39.122:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

--pod-network-cidr=192.168.0.0/16 is required by Calico (the CNI plugin we install in Step 11). If you choose a different CNI, this CIDR may differ. The `--apiserver-advertise-address` must be set to the node's IP that other nodes can reach — typically the private IP.

MASTER NODE — STEP 10

Configure kubectl Access

After `kubeadm init` completes, the cluster admin credentials are stored at `/etc/kubernetes/admin.conf`. To allow the current non-root user to run `kubectl` commands, you need to copy this configuration to the user's home directory.

Set up the kubeconfig for your user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Verify the master node appears (it will show NotReady until CNI is installed):

```
kubectl get nodes
```

Expected output at this stage: The master node will appear with a status of `NotReady`. This is normal — the node becomes Ready only after a CNI plugin is installed in the next step. The control plane components themselves are running correctly.

MASTER NODE — STEP 11

Install Calico Networking (CNI)

Calico is a widely-used, high-performance CNI plugin that provides pod networking and optional network policy enforcement. It uses BGP for routing pod traffic and integrates tightly with the Linux kernel's networking stack. We install it using the Tigera Operator approach for production-grade lifecycle management.

Install the Tigera Operator (manages Calico lifecycle):

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/tigera-operator.yaml
```

Download and apply the Calico custom resource configuration:

```
curl https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/custom-resources.yaml -O
kubectl apply -f custom-resources.yaml
```

Monitor pods until all are Running (this may take 1–2 minutes):

```
kubectl get pods -A
```

Once all Calico pods are Running, check node status:

```
kubectl get nodes
```

Expected result: The master node will transition from `NotReady` to `Ready` once Calico is fully deployed. The `--pod-network-cidr=192.168.0.0/16` used during init matches Calico's default configuration in `custom-resources.yaml` — do not change one without changing the other.

PART 2 — WORKER NODES

Worker Node Setup

All steps in Part 2 are performed on **each Worker Node** (Worker1 and Worker2). The prerequisites — Steps 1 through 8 — are identical to the Master Node setup. This ensures every node in the cluster has a consistent OS configuration, container runtime, and Kubernetes tooling before joining the cluster.

Worker Step 1

Disable Swap — same as Master Step 1

Worker Step 2

Load Kernel Modules — same as Master Step 2

Worker Step 3

Configure Networking — same as Master Step 3

Worker Step 4

Install containerd — same as Master Step 4

Worker Step 5

Install runc — same as Master Step 5

Worker Step 6

Install CNI Plugins — same as Master Step 6

Worker Step 7

Install Kubernetes Components — same as Master Step 7

Worker Step 8

Configure crictl — same as Master Step 8

After completing Steps 1–8 on each Worker Node, proceed to the Worker-specific steps below: joining the cluster and verifying membership.

WORKER NODE — STEP 9

Join Worker Node to the Cluster

This is the step that makes a Worker Node a member of your Kubernetes cluster. The `kubeadm join` command uses the token and certificate hash generated during `kubeadm init` to securely authenticate and register with the API server. Run this on **each Worker Node separately**.

Run the join command that was output by kubeadm init on the Master (replace values with your actual token and hash):

```
sudo kubeadm join 172.31.39.122:6443 \
  --token <your-token> \
  --discovery-token-ca-cert-hash sha256:<your-hash>
```

Token expiry: By default, kubeadm join tokens expire after 24 hours. If you need to add a worker node after the token has expired, generate a new one on the Master with: `kubeadm token create --print-join-command`. This prints a fresh, ready-to-use join command.

WORKER NODE — STEP 10

Verify Cluster Node Status

After running the join command on both worker nodes, switch back to the **Master Node** and verify that all nodes have successfully registered and are in a `Ready` state. This confirms that the control plane can communicate with each worker and that Calico networking is functioning across all nodes.

Run on the Master Node:

```
kubectl get nodes
```

Expected output:

```
NAME     STATUS   ROLES          AGE   VERSION
master   Ready    control-plane  10m   v1.34.x
worker1  Ready    <none>         5m    v1.34.x
worker2  Ref     <none>         4m    v1.34.x
```

If a node shows NotReady: Wait 1–2 minutes and check again — Calico may still be initializing on that node. If it persists, run `kubectl describe node <node-name>` and `journalctl -u kubelet -f` on the worker to diagnose the issue.

FINAL VERIFICATION

Full Cluster Pod Verification

The final check confirms that all system pods across every namespace are running correctly. A healthy cluster should show all pods in `Running` or `Completed` state — including the Calico networking pods, CoreDNS, kube-proxy, and all control plane component pods.

List all pods across all namespaces:

```
kubectl get pods -A
```

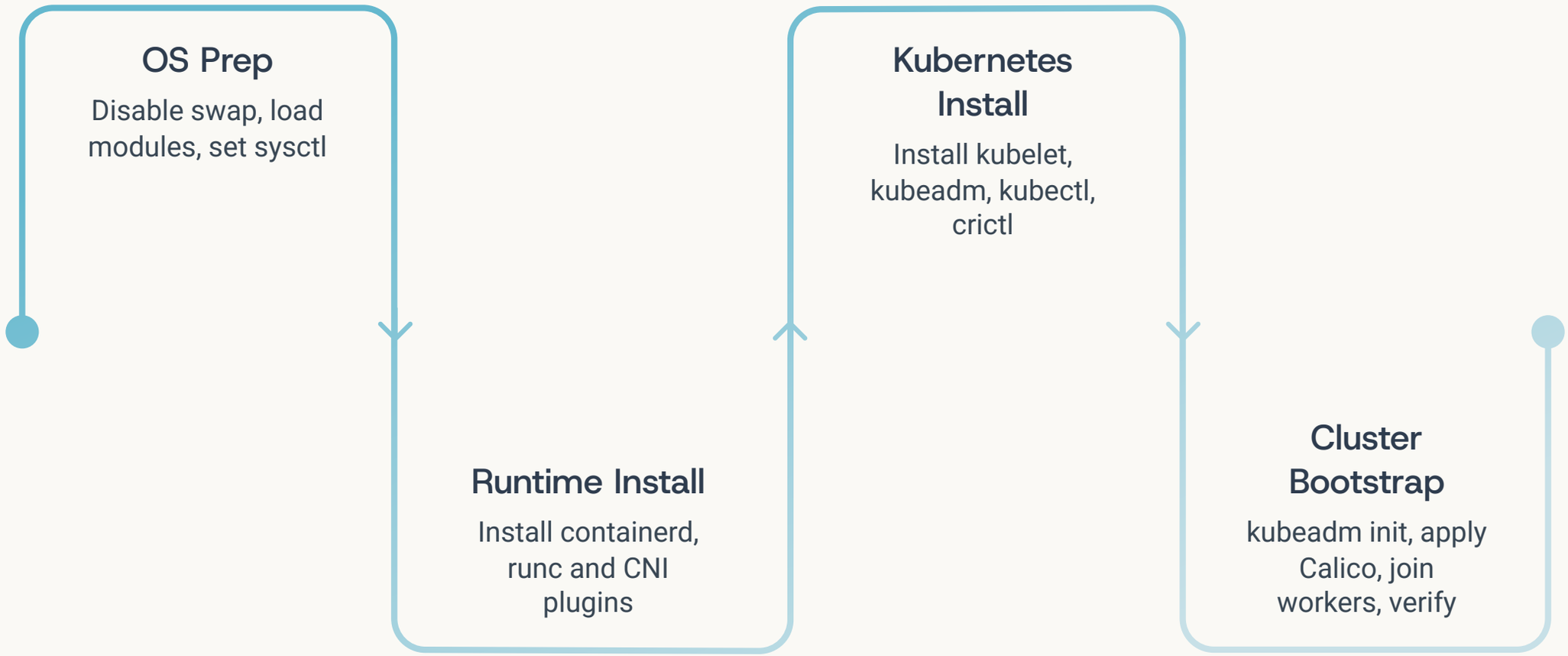
Expected healthy output will include pods similar to:

```
NAMESPACE        NAME                                READY   STATUS    RESTARTS
calico-system    calico-kube-controllers-xxx         1/1     Running   0
calico-system    calico-node-xxx (one per node)      1/1     Running   0
calico-system    calico-typha-xxx                    1/1     Running   0
kube-system      coredns-xxx                         1/1     Running   0
kube-system      etcd-master                         1/1     Running   0
kube-system      kube-apiserver-master               1/1     Running   0
kube-system      kube-controller-manager-master      1/1     Running   0
kube-system      kube-proxy-xxx (one per node)       1/1     Running   0
kube-system      kube-scheduler-master               1/1     Running   0
tigera-operator  tigera-operator-xxx                 1/1     Running   0
```

All systems green? If all pods show `Running` and all nodes show `Ready`, your Kubernetes cluster is fully operational and ready to deploy workloads. Congratulations — you have a production-grade, self-managed Kubernetes v1.34 cluster running on cloud VMs.

Complete Setup Flow: End-to-End Summary

Here is a consolidated view of every step executed across both the Master and Worker nodes, giving you a clear mental map of the entire installation process from OS preparation to a fully verified cluster.



The same OS preparation and runtime installation steps run on **all nodes**. The control plane initialization (`kubeadm init`) runs only on the Master. The cluster join (`kubeadm join`) runs only on Workers. CNI installation runs only on the Master — Calico then deploys its agents to all nodes automatically via DaemonSets.

Key Concepts Recap

kubeadm

Bootstrap tool that initializes and joins cluster nodes. Handles certificate generation, component startup, and kubeconfig creation automatically.

containerd + runc

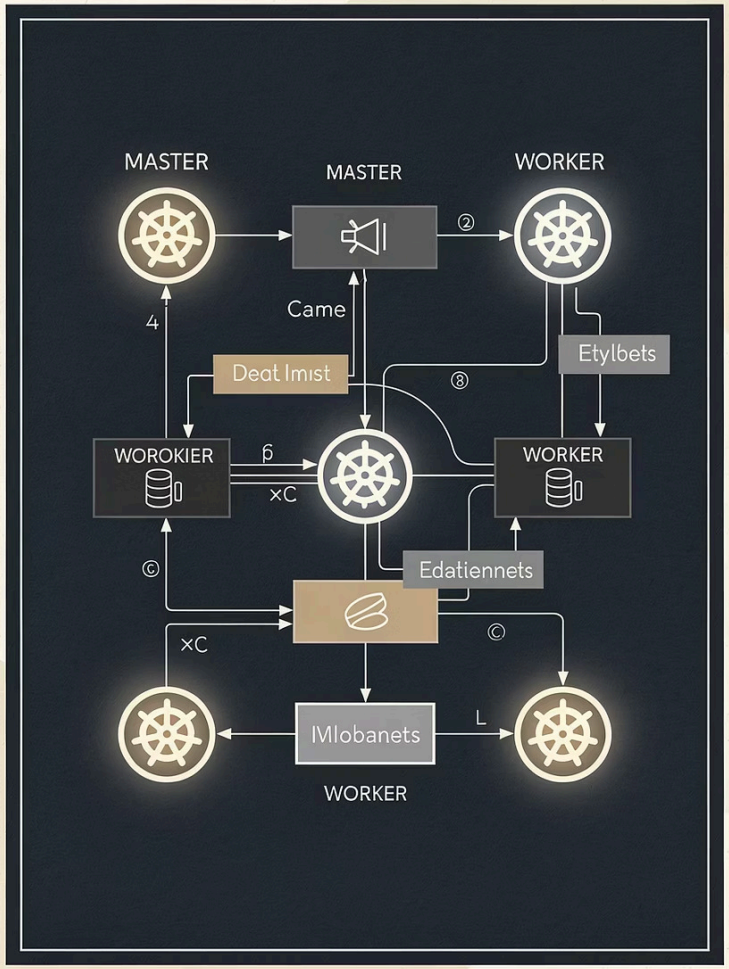
The container runtime stack. containerd manages the full lifecycle; runc is the low-level OCI runtime that actually spawns container processes using Linux namespaces and cgroups.

Calico CNI

Provides pod networking across all nodes using BGP routing. Required before any node transitions to Ready. Also supports Kubernetes NetworkPolicy for traffic control.

kubelet

The node agent running on every node. Watches the API server for pod assignments and ensures the correct containers are running and healthy at all times.



Troubleshooting Quick Reference

Even with careful execution, issues can arise. Here are the most common problems encountered during a kubeadm installation and the commands to diagnose them.

Node stuck in NotReady

Check kubelet logs: `journalctl -u kubelet -f`
Check node events: `kubectl describe node <name>`
Usually caused by CNI not yet running or swap still enabled.

Pods stuck in Pending or ContainerCreating

Check pod events: `kubectl describe pod <pod-name> -n <namespace>`
Often indicates a CNI networking issue or image pull failure.

kubeadm init fails

Run preflight checks: `sudo kubeadm init --dry-run`
Reset if needed: `sudo kubeadm reset`
Then re-run init after fixing the reported issue.

Join token expired

Generate a new token on Master: `kubeadm token create --print-join-command`
This outputs a complete, ready-to-use join command for new workers.

crictl connection refused

Verify containerd is running: `sudo systemctl status containerd`
Confirm socket path: `sudo crictl --runtime-endpoint unix:///var/run/containerd/containerd.sock ps`